

阶段 10：Agent 动态预算重分配实施方案

阶段 10：Agent 动态预算重分配实施方案

状态：Proposed

更新日期：2026-08-06

适用范围：Question Evolution Agent Harness 的搜索预算、生成预算、评分预算、分支预算和多 Agent 诊断预算

目标读者：实现 Planner、Replanner、预算治理、搜索调度和实验复盘的工程师

读完后应能做的事：明确什么是动态预算重分配、什么时候允许 Agent 提出调整、如何校验、如何落盘、如何测试。

1. 结论

动态预算重分配是指：实验运行到阶段边界后，Agent 根据中间观察，把剩余预算从低收益路径转移到高收益路径。

它不是让 Agent 修改已经消耗的预算，也不是让 Agent 绕过总预算继续探索。正确形式是：

代码块

- 1 Observation
- 2 -> Agent 生成预算调整建议
- 3 -> Budget Validator 校验
- 4 -> Replanner 生成下一步计划
- 5 -> Executor 执行通过校验的工具步骤
- 6 -> Budget Ledger 记录分配和消耗

一句话边界：**Agent 可以重新分配剩余预算，不能改历史消耗，不能突破硬预算，不能跳过校验和评分。**

2. 当前现状

当前项目已经有多类预算和停止条件：

当前能力	说明
boundary_target	目标边界候选数量。
max_search_steps	横向搜索最大步数。
request_budget	纵向搜索请求预算。
evaluation_budget	纵向搜索评分预算。
operator_plan	每个算子的 pending、running、completed 等状态。
termination_reason	记录预算耗尽、边界达到、候选耗尽等终止原因。
budget_warning	观察层可以识别预算或硬上限相关停止。

当前系统的问题不是没有预算，而是预算主要由固定搜索策略推进。运行中如果发现某个算子明显低收益，系统可以记录结果，但还没有形成统一的 Agent 预算调整建议、校验和落盘机制。

3. 动态预算重分配解决什么问题

动态预算重分配主要解决 4 类问题：

- 1. 低收益算子继续消耗预算，例如某算子连续产生 validation_failed 或 not_applicable。
- 2. 高收益算子预算不足，例如某算子已经出现稳定 score_decreased，但原计划没有给它继续探索空间。
- 3. 评分预算使用不合理，例如疑似有效候选没有重复评分，明显无效候选却继续评分。
- 4. 纵向叠加进入过早，例如第一层降分还不稳定，就直接进入第二层。

目标是减少无效探索，把剩余生成、评分、分支、搜索步数和模型调用资源转移到更有价值的路径。

4. 预算类型

本方案中的预算至少包括：

预算类型	含义	是否允许中途调整
生成预算	每个算子或分支还能生成多少候选	允许调整剩余部分。
候选预算	每轮最多保留多少候选进入校验和评分	允许调整剩余部分。
分支预算	每个算子还能开启多少分支	允许调整剩余部分。
搜索步数预算	本次搜索还能推进多少步	允许收缩，不能突破硬上限。
评分预算	还能进行多少次作答和评分	允许重分配，不能跳过必要评分。
重复评分预算	疑似有效或评分不稳定候选的复评次数	允许追加，但必须有证据。
纵向深度预算	是否进入下一层算子叠加	允许延后或取消，不能无条件加深。
模型调用预算	路由、生成、评分、分析的模型调用次数	允许按任务风险调整模型层级。
时间预算	单次实验或单个分支可用时间	允许提前停止低收益路径。

不允许调整的内容：已经发生的消耗、已发布的正式评分、已写入的正式状态和已冻结的快照。

5. 触发条件

只有在明确阶段边界或安全观察点才能触发预算重分配。

允许触发：

1. 一轮横向分支搜索完成后。
2. 一个算子的候选全部完成校验后。
3. 一批候选完成评分和效果分析后。
4. 纵向叠加第一层完成后。
5. 出现 `budget_warning`、`score_increased`、`validation_failed` 激增或 `not_applicable` 激增。
6. 多 Agent 复盘给出高成本低收益建议后。

禁止触发：

1. 单个工具正在运行中。
2. 评分尚未完成时直接根据候选表面难度调整。
3. manifest 损坏或产物缺失时继续分配预算。
4. 快照不一致时继续实验。

6. 输入观察

预算重分配不能只依靠 Agent 主观判断，必须读取结构化观察。

输入至少包括：

代码块

```
1  branch_status_counts
2  operator_plan_status
3  operator_attempt_count
4  validation_failed_count
5  not_applicable_count
6  score_increased_count
7  score_decreased_count
8  scoring_variance_summary
9  remaining_budget
10 termination_reason
11 evidence_refs
```

如果缺少评分结果，只能调整生成和校验预算，不能确认某条路径是高收益路径。

7. 输出结构

建议新增 `BudgetReallocationProposal`。

代码块

```
1  {"proposal_id": "budget_reallocation_001", "trigger":
   "round_completed", "summary": "018 出现稳定降分，016 validation_failed 较多，建议把
   剩余生成和评分预算转给 018", "current_budget": {"generation_remaining":
   12, "scoring_remaining": 18, "search_steps_remaining": 8}, "changes": [{"target":
   "operator:016_close_alternative_normalization", "action":
   "reduce", "budget_type": "generation", "from": 4, "to": 0, "reason": "连续
   validation_failed", "evidence_refs": ["round_1/branch_results.jsonl#branch-
   016"]}, {"target": "operator:018_baseline_scope_mismatch", "action":
   "increase", "budget_type": "generation", "from": 3, "to": 7, "reason": "已出现
   score_decreased 候选", "evidence_refs": ["round_1/branch_results.jsonl#branch-
   018"]}], "requires_validator": true, "forbidden_actions_requested": []}
```

输出要求：

1. 每个调整必须说明来源、目标、预算类型、调整前后数值和证据引用。
2. 不能只写“增加 O18 预算”，必须写清楚从哪里回收、加到哪里、加多少。
3. 没有证据引用的调整只能进入 `needs_human_review`。

8. 预算调整规则

8.1 可以减少预算的情况

满足以下任一条件，可以减少该路径的剩余预算：

1. 同一算子连续产生 `validation_failed`。
2. 同一算子连续产生 `not_applicable`。
3. 分支耗时高且没有 `score_decreased` 或高价值候选。
4. 候选重复率高，命中已有 `seen_prompt_hashes`。
5. 评分结果显示 `score_increased`，且不是评分波动。
6. 该路径已经达到本轮局部目标。

8.2 可以增加预算的情况

满足以下任一条件，可以增加该路径的剩余预算：

1. 已出现 `score_decreased`，且可回答性校验通过。
2. 出现接近边界的 `score_unchanged`，但机制命中较好。
3. 多个候选指向同一稳定失败机制。
4. 评分分歧较大，需要追加重复评分确认。
5. 第一层纵向候选有效，但第二层推理压力尚未测试。

8.3 必须进入人工复核的情况

以下情况不能自动调整为继续探索，必须进入人工复核或只读复盘：

1. `score_decreased` 候选存在泄漏风险。
2. 候选可回答性不确定。
3. 评分结果波动大，无法判断真实收益。
4. 调整会导致某类算子完全被排除。
5. 调整依赖单次实验事实，且会影响后续全局策略。

9. 校验规则

新增 `BudgetValidator`，负责拒绝不安全调整。

必须校验：

1. 调整后总预算不能超过原始硬预算。
2. 已消耗预算不能被改写。
3. 评分预算不能被挪走到导致有效候选无法评分。
4. `score_increased` 路径不能在同条件下继续追加预算。
5. 未通过校验的候选不能获得评分预算。
6. 纵向叠加追加预算前，父题必须可回答，第一层不能有负收益。
7. 调整不能改变已冻结的 `operator_plan_revision`；如需改变，必须生成新的计划版本。
8. 调整不能删除历史状态和记忆。

校验失败时，输出 `rejected_by_budget_validator`，并写明拒绝原因。

10. 执行方式

动态预算重分配不直接修改当前工具内部状态。正确执行方式是生成新的计划版本。

代码块

```
1  current_plan_r001
2  -> observation
3  -> budget_reallocation_proposal
4  -> budget_validator
5  -> replan
6  -> current_plan_r002
```

执行要求：

1. 原计划保留，不覆盖。
2. 新计划写明替代哪个计划版本。
3. 新计划只影响后续未执行步骤。
4. 已完成分支不回写预算。
5. 报告中必须展示预算调整前后差异。

11. 和现有搜索状态的关系

当前搜索状态中的 `operator_plan`、`search_state` 和 `vertical_search_state` 仍是正式执行状态。Agent 的预算调整建议不能直接手写这些状态。

正确关系是：

代码块

```
1 search_state 提供事实
2 Agent 生成预算建议
3 BudgetValidator 校验建议
4 Replanner 生成新计划
5 Executor 通过正式工具推进 search_state
```

如果需要改变某个算子的后续分配，应通过新计划参数、工具参数或正式调度入口实现，不能让 Agent 直接改 JSONL 中的状态字段。

12. 多 Agent 参与方式

动态预算重分配可以使用专项智能体辅助，但最终建议必须由主控合成。

建议职责：

专项智能体	职责	输出
搜索成本智能体	找出高成本低收益分支	成本风险和停止建议。
算子诊断智能体	判断低收益是否来自算子不适配	算子风险和证据引用。
评分稳定性智能体	判断是否需要追加重复评分	复评建议。
预算合成智能体	合成预算调整草案	BudgetReallocationProposal。

限制：专项智能体只给建议，不直接改变预算、不直接执行工具、不直接写正式搜索状态。

13. 建议文件结构

后续实现可新增：

代码块

```
1 agent_runtime/budgeting/
2   __init__.py
3   budget_state.py
4   budget_observer.py
5   budget_reallocator.py
6   budget_validator.py
7   budget_report.py
8
9   schemas/
10    budget_state.schema.json
11    budget_reallocation_proposal.schema.json
12    budget_reallocation_decision.schema.json
13
```

```
14 tests/
15     test_budget_reallocation.py
16     test_budget_validator.py
17     test_budget_replan_integration.py
18     test_budget_report.py
```

模块职责：

1. `budget_state.py` 记录初始预算、已消耗预算和剩余预算。
2. `budget_observer.py` 从观察结果提取预算信号。
3. `budget_reallocator.py` 生成预算调整建议。
4. `budget_validator.py` 校验预算建议是否合法。
5. `budget_report.py` 把预算调整写入报告。

14. 实施步骤

14.1 第一步：预算账本

实现内容：新增预算账本，记录初始预算、消耗、剩余和硬上限。

验收标准：每次工具调用后都能看到预算消耗；预算耗尽有明显原因。

14.2 第二步：预算观察

实现内容：从分支结果、搜索状态、纵向状态和评分结果中提取预算信号。

验收标准：能识别高成本低收益、评分不稳定、连续无效生成和预算告警。

14.3 第三步：预算调整建议

实现内容：生成 `BudgetReallocationProposal`，包含调整前后数值和证据引用。

验收标准：没有证据的调整不会进入自动执行链路。

14.4 第四步：预算校验器

实现内容：实现 `BudgetValidator`，拒绝突破硬预算、改写历史消耗、跳过评分和直接修改状态的建议。

验收标准：越权预算调整会被拒绝并写入事件。

14.5 第五步：重规划接入

实现内容：通过 Replanner 生成新计划版本，只影响后续未执行步骤。

验收标准：新计划能说明替代关系，旧计划和历史消耗保持不变。

14.6 第六步：报告和复盘

实现内容：报告展示预算调整原因、证据、调整前后数值、校验结果和实际收益。

验收标准：人工 reviewer 能判断本次预算调整是否合理。

15. 必测场景

必须覆盖以下测试：

1. O16 连续 `validation_failed`，剩余生成预算被收缩。
2. O18 出现稳定 `score_decreased`，获得追加生成预算。
3. 疑似有效候选评分分歧大，追加重复评分预算。
4. `score_increased` 路径请求同条件继续探索，必须拒绝。
5. 已消耗预算被修改，必须拒绝。
6. 调整后超过硬预算，必须拒绝。
7. 未通过校验候选请求评分预算，必须拒绝。
8. 纵向叠加前父题不可回答，必须拒绝追加深度预算。
9. 预算调整没有证据引用，必须进入 `needs_human_review`。
10. 新计划只影响未执行步骤，不能覆盖旧计划。
11. 专项智能体请求直接修改 `search_state`，必须拒绝。
12. 报告必须展示预算调整前后差异。

16. 最终验收标准

本阶段完成后，应满足：

1. Agent 能根据观察结果提出预算重分配建议。
2. 所有建议都有证据引用、调整前后数值和目标路径。
3. Budget Validator 能拒绝越权调整。
4. 已消耗预算和历史状态不会被改写。
5. 新预算只通过新计划版本影响后续步骤。
6. 动态预算调整不能跳过校验、评分、manifest 和状态更新。
7. 报告能清楚解释为什么收缩某条路径、为什么追加另一条路径。
8. 多 Agent 可以辅助诊断，但不能直接改变正式预算和搜索状态。

17. 暂不实施内容

以下内容不进入本阶段：

1. Agent 直接修改 `search_state`、`operator_plan` 或 `vertical_search_state`。
2. 根据单次降分自动提高全局默认预算。
3. 让预算调整绕过 Plan Validator。
4. 让低成本搜索智能体直接决定预算分配。
5. 根据缓存命中率自动增加实验预算。
6. 在工具运行中强行抢占或终止正在执行的子进程。

这些内容风险高，容易破坏当前项目的可恢复、可复现和可审计边界。